42. Trapping Rain Water

Hard · Array · Two Pointers · Stack · Dynamic Programming

Approach — Two Pointers with Layer-by-layer Water Accounting

Instead of computing trapped water column by column, this solution thinks in **horizontal layers**. Two pointers start from both ends and move inward. At each step, the bottleneck height — the minimum of `height[l]` and `height[r]` — determines how much water the current layer between the two pointers can hold.

The variable `prev` tracks the water level of the last processed layer. When the new bottleneck `mnHeight` rises above `prev`, a new layer of water is added for all `r-l-1` cells between the pointers. When it does not rise, the current bottleneck wall is too short to hold water at this level, so `mnHeight` is subtracted from the answer to account for the space occupied by that wall itself.

Key Insight: Water can only be trapped up to the height of the shorter boundary wall. By always moving the pointer on the shorter side inward, we guarantee that the taller side remains a valid boundary for any future water layers between the pointers.

Step-by-step Breakdown

Step 1 — Initialize pointers and trackers

Set `l = 0` and `r = height.size()-1` at both ends. `prev = 0` tracks the last settled water layer height. `ans` accumulates the total trapped water, starting with the full area between the pointers and subtracting walls as they are encountered.

Step 2 — Compute the bottleneck height

At each iteration, `mnHeight = min(height[l], height[r])` gives the maximum water level that can be held at this layer — bounded by the shorter of the two current walls.

Step 3 — Add a new water layer or subtract a wall

If `mnHeight > prev`, a new higher layer exists. Add the water volume for all `r-l-1` interior cells at this layer: `(r-l-1)*(mnHeight-prev)`. First subtract `prev` from ans to remove the previous layer's contribution, then add the new full layer, and update `prev = mnHeight`. If `mnHeight <= prev`, the current wall is at or below the existing water level and simply occupies space — subtract `mnHeight` to account for the wall's footprint.

Step 4 — Advance the shorter pointer

Move whichever pointer points to the shorter wall: if `mnHeight == height[l]`, advance `l++`; otherwise advance `r--`. This ensures the taller wall is always kept as a boundary while the search continues inward.

Solution Code

```cpp
class Solution {
public:
    int trap(vector<int>& height) {
        int l = 0, r = height.size()-1, prev = 0, ans = 0;
        while(l < r)
        {
            int mnHeight = min(height[l], height[r]);
            if(mnHeight > prev)
            {
                ans -= prev;
                ans += (r-l-1)*(mnHeight-prev);
                prev = mnHeight;
            }
            else ans -= mnHeight;

            if(mnHeight == height[l]) l++;
            else r--;
        }
        return ans;
    }
};
```

Complexity Analysis

TIME COMPLEXITY	SPACE COMPLEXITY
O(n)	**O(1)**

Each element is visited at most once as the two pointers converge. The entire array is processed in a single pass.	Only a constant number of variables are used. No auxiliary arrays or stacks are needed — the computation is fully in-place.